Ramneet Kaur
230253

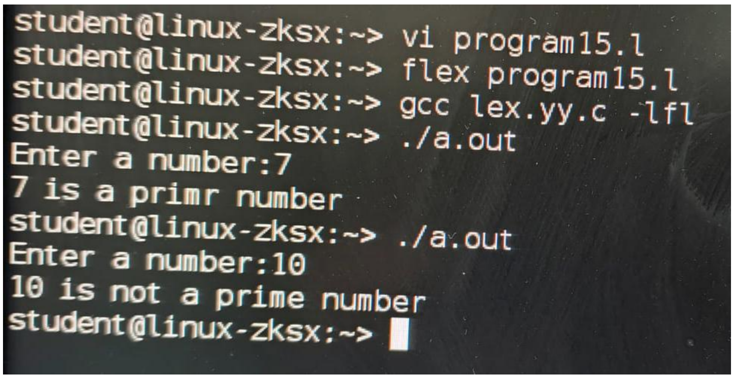
PROGRAM 14 – Write a program using Flex to check whether the given number is prime or not.

CODE –

```
%{
#include <stdio.h>
#include <stdlib.h>

int isPrime(int n) {
int i;
    if (n <= 1) return 0;
    for ( i = 2; i * i <= n; i++) {
        if (n % i == 0)
            return 0;
    }
    return 1;
}
%}

%%

[0-9]+ {
            int num = atoi(yytext);
            if (isPrime(num))
                printf("%d is a PRIME number\n", num);
            else
                printf("%d is NOT a prime number\n", num);
        }

\n      return 0;

.       ;   /* Ignore other characters */
```

```
%%

int yywrap() {
    return 1;
}

int main() {
    printf("Enter a number: ");
    yylex();
    return 0;
}
```

OUTPUT –



Ramneet Kaur
230253

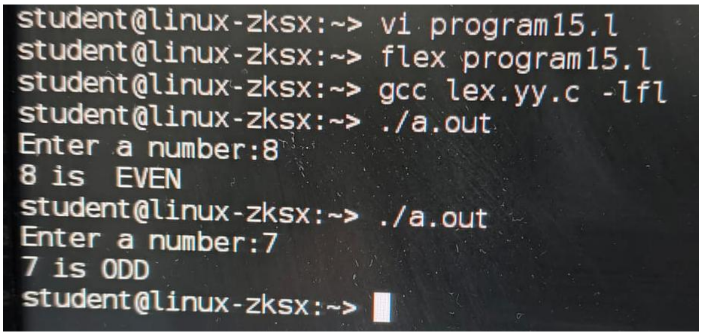
PROGRAM 15 – Write a program using Flex to check whether the given number is odd or even.

CODE –

```
%{
#include <stdio.h>
#include <stdlib.h>
%}

%%

[0-9]+ {
            int num = atoi(yytext);
            if (num % 2 == 0)
                printf("%d is EVEN\n", num);
            else
                printf("%d is ODD\n", num);
        }

\n      return 0;

.       ;   /* Ignore other characters */

%%

int yywrap() {
    return 1;
}

int main() {
    printf("Enter a number: ");
    yylex();
```

```
    return 0;
}
```

OUTPUT –



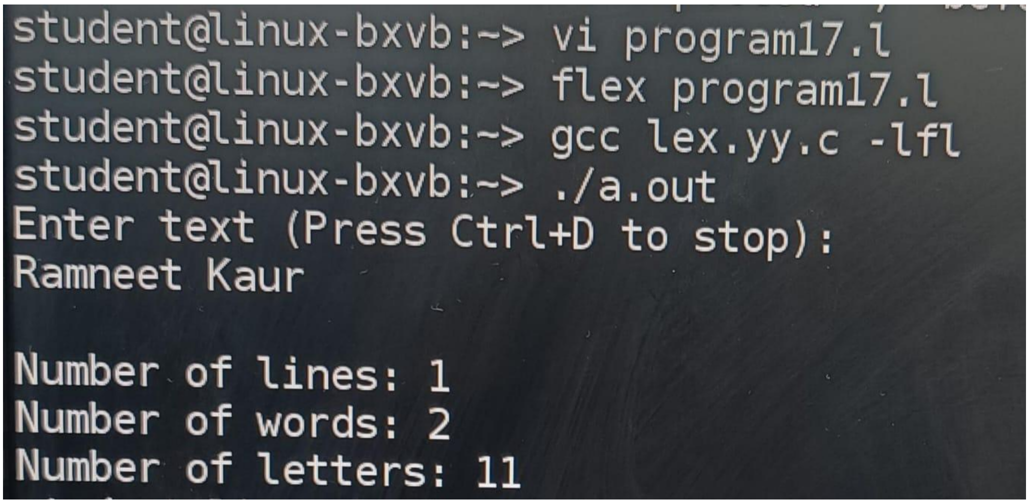
Ramneet Kaur
230253

PROGRAM 16 – Write a program using Flex to count the number of lines, words and letters.

CODE –

```
%{
#include <stdio.h>

int lines = 0;
int words = 0;
int letters = 0;
%}

%%
[a-zA-Z]    { letters++; }
[ \t]+      { words++; }
\n          { lines++; words++; }
.           { }
%%

int yywrap() {
   return 1;
}
int main() {
   printf("Enter text (Press Ctrl+D to stop):\n");
   yylex();

   printf("\nNumber of lines: %d\n", lines);
   printf("Number of words: %d\n", words);
   printf("Number of letters: %d\n", letters);

   return 0;
}
```

OUTPUT –


student@linux-bxvb:~> vi program17.l
student@linux-bxvb:~> flex program17.l
student@linux-bxvb:~> gcc lex.yy.c -lfl
student@linux-bxvb:~> ./a.out
Enter text (Press Ctrl+D to stop):
Ramneet Kaur
Number of lines: 1
Number of words: 2
Number of letters: 11

Ramneet Kaur
230253

PROGRAM 17 – Write a program using Flex to Read and copy the content of a file to another file.

CODE –

```
%{
#include <stdio.h>
#include <string.h>
FILE *yyin, *yyout;
char line[100];
%}
%%
\n          { fprintf(yyout, "%s\n", line); }
(.|\t)+     { strcpy(line, yytext); fprintf(yyout, "%s", line); }
%%
int yywrap() {
    return 1;
}
int main() {
    yyin = fopen("cd1.txt", "r");   // input file
    yyout = fopen("cd2.txt", "w");  // output file

    if (!yyin || !yyout) {
        printf("File error\n");
        return 1;
    }
    yylex();
    fclose(yyin);
    fclose(yyout);
    printf("File copied successfully!\n");
    return 0;
}
```

OUTPUT –

```
student@linux:~$ vi cd16.l
student@linux:~$ flex cd16.l
student@linux:~$ gcc lex.yy.c -lfl -std-c999
student@linux:~$ ./a.out
student@linux:~$ vi cd2.txt
Hello World,
How are you?
This is compiler design lab.
~
student@linux:~$ vi cd2.txt
Hello World,
How are you?
This is compiler design lab.
~
```

Ramneet Kaur
230253

PROGRAM 18 – Write a program using Flex to perform all the operations of a calculator using atof function.

CODE –

```
%{
#include <stdio.h>
#include <stdlib.h>
float result = 0;
%}
%%
[0-9]+(\.[0-9]+)?   { result = atof(yytext); }
[+]              {
                   float num;
                   scanf("%f", &num);
                   result += num;
                   printf("Result = %f\n", result);
                 }
[-]              {
                   float num;
                   scanf("%f", &num);
                   result -= num;
                   printf("Result = %f\n", result);
                 }
[*]              {
                   float num;
                   scanf("%f", &num);
                   result *= num;
                   printf("Result = %f\n", result);
                 }
[/]              {
                   float num;
                   scanf("%f", &num);
```

```
            if(num != 0)
                result /= num;
            else
                printf("Division by zero error\n");
            printf("Result = %f\n", result);
        }
\n          return 0;
.           ;
%%
int yywrap() {
    return 1;
}
int main() {
    printf("Enter expression (e.g. 5 + 3):\n");
    yylex();
    return 0;
}
```

OUTPUT –

```
student@linux:~$ vi program19.l
student@linux:~$ flex program19.l
student@linux:~$ gcc lex.yy.c -lfl -std=c99
student@linux:~$ ./a,out
student@linux:~$ ./a.out
Enter expression (e.g. 5 + 3):
10 - 3
Result = 8.000000
student@linux:~$ ../a.out
Enter expression (e.g. 5 + 3):
6 + 4
Result = 12.000000
student@linux:~$ ../a.out
Enter expression (e.g. 5 + 3):
6 + 2
Result = 4.0000000
student@linux:~$ ../a.out
Enter expression (e.g. 5 + 3):
8 / 2
Result = 4.000000
student@linux:~$
```

Ramneet Kaur
230253

PROGRAM 19 – Write a program using Flex to demonstrate the following: ECHO, REJECT, YYLESS, YYMORE, START conditions.

CODE –

```
%{
#include <stdio.h>
%}
%x COMMENT   /* start condition */
%%
/* START CONDITION (ignore comments) */
"//".*          { BEGIN(COMMENT); }
<COMMENT>\n     { BEGIN(INITIAL); }
<COMMENT>.      ;
/* REJECT example */
"ab"            { printf("Found AB\n"); REJECT; }
/* YYLESS example */
"hello"         {
                  yyless(3);
                  printf("YYLESS Output: %s\n", yytext);
                }
/* YYMORE example */
"a"             { yymore(); }
"b"             { printf("YYMORE Output: %s\n", yytext); }
/* ECHO example */
[a-zA-Z]+       { ECHO; }
/* Default rules */
\n              { printf("\n"); }
.               { ECHO; }
%%
int yywrap() {
   return 1;
}
```

```
int main() {
    printf("Enter input:\n");
    yylex();
    return 0;
}
```

OUTPUT –

```
student@linux:~$ vi program20.l
student@linux:~$ flex program20.l
student@linux:~$ gcc lex.yy.c -lfl -std-c99
student@linux:~$ ./a.out
Enter input:
hello ab a a b //this is comment
YYLESS Output: hel
Found AB
ab
YYMORE Output: ab
student@linux:~$ ./a.out
Enter expression (e.g. 5 + 3):
10 - a
Result = G.000000
student@linux:~$ ./a.out
~
~
```